

SCS 2312

Computational Models and Programming Language Concepts

Statement Level Control Structures

Tharindu Wijethilake

tnb@ucsc.cmb.ac.lk



UNIVERSITY OF COLOMBO SCHOOL OF COMPUTING



Plan for the day

- Control Structures
- Selection Statements
- Iterative Statements
- Unconditional Branching

Control Structures

- Control statements allow the execution flow to jump, repeat, or skip code
- Design issue
 - Should the control structure have multiple entries?

Spaghetti Code

- Tangled, complex, and disorganised structure of the source code.
- Early programming languages used GOTO statements



Edgar Dijkstra: Go To Statement Considered Harmful

Go To Statement Considered Harmful

Key Words and Phrases: go to statement, jump instruction, branch instruction, conditional clause, alternative clause, repetitive clause, program intelligibility, program sequencing

CR Categories: 4.22, 5.23, 5.24

EDITOR:

For a number of years I have been familiar with the observation that the quality of programmers is a decreasing function of the density of **go to** statements in the programs they produce. More recently I discovered why the use of the **go to** statement has such

dynamic progress is only chaotic. The call of the procedure we refer to is not a call, we can characterize the program by its textual indices, the length of the procedure, the dynamic depth of procedure.

Let us now consider repetition (or **repeat A until B**). Logic is superfluous, because we can replace recursive procedures. For recursion, we include them: on the one hand, it is not mented quite comfortably with

<https://homepages.cwi.nl/~storm/teaching/reader/Dijkstra68.pdf>

Spaghetti Code

```
1  #include <stdio.h>
2
3  int main() {
4      int myinput = 0;
5
6      printf("Enter a number 0 or 1: ");
7      scanf("%d", &myinput);
8
9      if (myinput == 0)
10         goto input0;
11
12     printf("You entered: %d which is higher than 0\n", myinput);
13
14
15     input0:
16     printf("Label input0: %d\n", myinput);
17
18     return 0;
19 }
```

```
Enter a number 0 or 1: 0
Label input0: 0
```

```
Enter a number 0 or 1: 1
You entered: 1 which is higher than 0
Label input0: 1
```

Spaghetti Code

- Problems
 - Difficult to debug
 - Changing one line of code might break a seemingly unrelated part of the program because of hidden dependencies.
 - Low reusability
 - Poor readability

Spaghetti Code

- How to solve this problem
 - Structured Control
 - Selection (if-then-else)
 - Iteration (loops)

Selection Statements

- Choosing between two or more execution paths
- Selection statements categories
 - Two-way
 - n-way (multiple selection)

Selection Statements

- Selection statements categories
 - Two-way
 - Design issues
 - What is the form and type of the expression that controls the selection?
 - How are the `then` and `else` clauses specified?
 - How should the meaning of nested selectors be specified?

Selection Statements

- Selection statements categories
 - Two-way

```
if control_expression  
then clause  
else clause
```

```
<if_stmt> → if <logic_expr> then <stmt>  
           | if <logic_expr> then <stmt> else <stmt>
```

Selection Statements

- Selection statements categories
 - Two-way
 - Nesting Selectors - Issues?

```
if (sum == 0)
    if (count == 0)
        result = 0;
else
    result = 1;
```

Selection Statements

```
if (sum == 0) {  
    if (count == 0)  
        result = 0;  
}  
else  
    result = 1;
```

```
if (sum == 0) {  
    if (count == 0) {  
        result = 0;  
    }  
    else {  
        result = 1;  
    }  
}
```

```
if (sum == 0) {  
    if (count == 0) {  
        result = 0;  
    }  
} else {  
    result = 1;  
}
```

Selection Statements

```
if a > b then sum = sum + a  
    acount = acount + 1  
else sum = sum + b  
    bcount = bcount + 1  
end
```

```
if sum == 0 then  
    if count == 0 then  
        result = 0  
    else  
        result = 1  
    end  
end
```

```
if sum == 0 :  
    if count == 0 :  
        result = 0  
else:  
    result = 1
```

Selection Statements

- Selection statements categories
 - n-way (multiple selection)
 - Selection of one of any number of statements or statement groups
 - Design issues
 - Is execution flow through the structure restricted to include just a single selectable segment?
 - How are the case values specified?
 - How should unrepresented selector expression values be handled, if at all?

Selection Statements

- Selection statements categories
 - n-way (multiple selection)

```
switch (expression) {  
    case constant_expression1 : statement1;  
    . . .  
    case constantn : statementn;  
    [default : statementn+1]  
}
```

Selection Statements

- Selection statements categories
 - n-way (multiple selection)

```
switch (index) {  
    case 1:  
    case 3: odd += 1;  
           sumodd += index;  
           break;  
    case 2:  
    case 4: even += 1;  
           sumeven += index;  
           break;  
    default: printf("Error in switch, index = %d\n", index);  
}
```

Selection Statements

- Selection statements categories
 - n-way (multiple selection)

```
if count < 10 :  
    bag1 = True  
elif count < 100 :  
    bag2 = True  
elif count < 1000 :  
    bag3 = True
```

```
if count < 10 :  
    bag1 = True  
else :  
    if count < 100 :  
        bag2 = True  
    else :  
        if count < 1000 :  
            bag3 = True  
        else :  
            bag4 = True
```

Selection Statements

```
1  #include <stdio.h>
2
3  int main() {
4      int value = 2;
5
6      switch (value) {
7          case 1:
8              printf("A\n");
9              break;
10         case 2:
11             printf("B\n");
12             break;
13         case 3:
14             printf("C\n");
15             break;
16         case 4:
17             printf("D\n");
18             break;
19         default:
20             printf("Invalid value\n");
21     }
22     return 0;
23 }
```

```
1  #include <stdio.h>
2
3  int main() {
4      int score = 85;
5
6      if (score >= 90) {
7          printf("Grade: A\n");
8      } else if (score >= 80) {
9          printf("Grade: B\n");
10     } else if (score >= 70) {
11         printf("Grade: C\n");
12     } else if (score >= 60) {
13         printf("Grade: D\n");
14     } else {
15         printf("Grade: F\n");
16     }
17
18     return 0;
19 }
```

Iterative Statements

- Repeated execution of a statement or compound statement
 - Design issues
 - How is the iteration controlled?
 - Where should the control mechanism appear in the loop statement?

Iterative Statements

- Counter-Controlled Loops
 - Loop variable - maintain the count value
 - Loop parameters - initial value, terminal value and step size

Iterative Statements

- Counter-Controlled Loops
 - Design issues
 - What are the type and scope of the loop variable?
 - Should it be legal for the loop variable or loop parameters to be changed in the loop, and if so, does the change affect loop control?
 - Should the loop parameters be evaluated only once, or once for every iteration?

Iterative Statements

- Counter-Controlled Loops

```
for (expression_1; expression_2; expression_3)
```

```
    expression_1  
loop:  
    if expression_2 = 0 goto out  
    [loop body]  
    expression_3  
    goto loop  
out: . . .
```

```
for (int i = 1; i <= 5; i++)  
{  
    printf("Iteration number: %d\n", i);  
}
```

Concepts of Programming Languages, Robert W. Sebesta

Iterative Statements

- Logically-Controlled Loops
 - Repetition control is based on a **Boolean expression** rather than a counter
 - Pre-test: Condition is checked before the body
 - Post-test: Condition is checked after the body

Iterative Statements

- Logically-Controlled Loops
 - Design issues
 - Should the control be pre-test or post-test?
 - Should the logically controlled loop be a special form of a counting loop or a separate statement?

Iterative Statements

- Logically-Controlled Loops
 - Pre-test

```
1  #include <stdio.h>
2
3  int main() {
4      int value = 0;
5
6      while (value < 5) {
7          value++;
8          printf("Iteration Number %d\n", value);
9      }
10
11     printf("Loop Completed\n");
12     return 0;
13 }
```

Iterative Statements

- Logically-Controlled Loops
 - Post-test

```
1  #include <stdio.h>
2
3  int main() {
4      int value = 0;
5
6      do {
7          value++;
8          printf("Iteration Number %d\n", value);
9      } while (value < 5);
10
11     printf("Loop Completed\n");
12     return 0;
13 }
```

Iterative Statements

- Iteration Based on Data Structures
 - User defined data structure
 - Iterator returns an element from a particular data structure in some specific order

Iterative Statements

- Iteration Based on Data Structures

```
1  fruits = ["apple", "banana", "pineapple"]
2
3  ▼ for fruit in fruits:
4      print(f"{fruit}")
5
```

```
1  #include <stdio.h>
2
3  int main() {
4      int numbers[] = {10, 20, 30, 40, 50};
5      int length = 5;
6
7      for (int i = 0; i < length; i++) {
8          printf("Value: %d\n", numbers[i]);
9      }
10     return 0;
11 }
```

Iterative Statements

- User-Located Loop Control Mechanisms
 - Allow the programmer to decide exactly where the loop should terminate
 - `break`
 - Immediately terminates the loop and transfers control to the statement following the loop
 - `continue`
 - Skips the remainder of the current iteration and jumps straight to the next evaluation of the loop control expression

Iterative Statements

- User-Located Loop Control Mechanisms

```
1  #include <stdio.h>
2
3  int main(){
4
5      for (int value = 1; value <= 10; value++) {
6          if (value == 5) {
7              break;
8          }
9          printf("Iteration Number: %d \n", value);
10     }
11     return 0;
12 }
```

```
Iteration Number: 1
Iteration Number: 2
Iteration Number: 3
Iteration Number: 4
```

Iterative Statements

- User-Located Loop Control Mechanisms

```
1  #include <stdio.h>
2
3  int main(){
4
5      for (int value = 1; value <= 10; value++) {
6          if (value == 5) {
7              continue;
8          }
9          printf("Iteration Number: %d \n", value);
10     }
11     return 0;
12 }
```

```
Iteration Number: 1
Iteration Number: 2
Iteration Number: 3
Iteration Number: 4
Iteration Number: 6
Iteration Number: 7
Iteration Number: 8
Iteration Number: 9
Iteration Number: 10
```

Iterative Statements

- User-Located Loop Control Mechanisms
 - Some can argue that these are like GOTO statements
 - Multiple exits

Unconditional Branching

- Control statement that transfers execution to a different part of the program without testing any condition

Unconditional Branching

```
1  #include <stdio.h>
2
3  int main() {
4      int i = 0;
5
6      start_loop:
7          if (i >= 3) goto end_loop;
8          printf("i = %d\n", i);
9          i++;
10         goto start_loop;
11
12     end_loop:
13         printf("Done.\n");
14         return 0;
15 }
```

```
i = 0
i = 1
i = 2
Done.
```

Unconditional Branching

- Modern languages have completely removed the GOTO statement
- They use `break` and `continue`
 - Still works like GOTO but they tied to the loop

Guarded Commands